

Authentication

Systems Security Fundamentals

A. Shapoval

April 28, 2026

Three Key Components of Security

- ➊ **Authentication** — identify the principal performing an action
- ➋ **Authorization** — determine who is allowed to do what
- ➌ **Enforcement** — ensure only authorized principals perform specific actions

- **Principal:** an entity associated with a security identifier
- Principals are authorized to perform certain actions

Definition

Authentication is the process by which an entity proves to a computer that it is a particular principal.

Basic idea:

- 1 Computer believes the principal knows a secret
- 2 Entity proves knowledge of the secret
- 3 Therefore, the computer believes the entity is the principal

Local Authentication: Passwords

- Most common authentication mechanism: **passwords**
- Main advantage:
 - Convenient and easy to deploy
- Main disadvantage:
 - Often insecure

“Humans are incapable of securely storing high-quality cryptographic keys, and they have unacceptable speed and accuracy when performing cryptographic operations.”

— Kaufman, Perlman, and Speciner,
Private Communication in a Public World (1995)

The Fundamental Password Problem

Core Issue

Passwords are easy to guess.

- Secure passwords must be:
 - long
 - obscure
 - difficult to predict
- However:
 - short passwords are easy to crack
 - long passwords are difficult to remember
 - users often write long passwords down

Technology vs Password Strength

- Increasing computational power requires stronger passwords
- Original UNIX systems used:
 - 5-letter lowercase passwords

Exhaustive Search Space

$$26^5 \approx 10\,000\,000$$

possible passwords

Year	Time to Crack Password Space
1975	$\approx$ 1 day
1992	$\approx$ 10 seconds
2026	$\approx$ 0.1 seconds
2050	?

Weak Password Choices

- Many users choose highly predictable passwords
- Common examples:
 - English words
 - names of people or places
 - fictional characters
 - astronomy-related names
- Typical variations:
 - reversing words
 - replacing letters with numbers
 - i $\rightarrow$ 1
 - e $\rightarrow$ 3

Observation

Even “modified” dictionary words are often vulnerable to modern password cracking techniques.

Takeaways

- Authentication is a foundational security mechanism
- Passwords remain widely used despite major weaknesses
- Advances in computing continuously weaken old password assumptions
- Human behavior is often the weakest link

Reality

Passwords are imperfect, but careful system design can make attacks significantly harder.

- 1 Enforce stronger password quality
- 2 Avoid storing plaintext passwords
- 3 Slow down password guessing
- 4 Use password salting and hashing
- 5 Design secure password reset procedures
- 6 Pay attention to implementation details

Enforcing Password Quality I

Modern systems often require passwords with:

- minimum length (e.g., 12–16 characters)
- upper- and lowercase letters
- digits
- special characters

Password Search Space

A password using 70 possible symbols and length 8:

$$70^8 \approx 5 \times 10^{14}$$

possible combinations

Online password validation:

Enforcing Password Quality II

- reject dictionary words
- reject common passwords
- prevent predictable patterns
- estimate password entropy

The Problem with Complexity Rules I

Users Adapt Predictably

Even with password policies, many users still choose weak patterns.

Examples:

- Summer2025!
- Password1!
- 7 lowercase letters + 1 digit + 1 punctuation symbol

Modern recommendation:

- Prefer *long passphrases* over complex short passwords
- Example:
 - correct-horse-battery-staple

The Problem with Complexity Rules II

NIST and modern security guidelines now emphasize:

- password length
- screening against known leaked passwords
- usability

Never Store Plaintext Passwords I

Critical Principle

A system should never store user passwords in plaintext.

Instead:

- store only a cryptographic hash of the password
- compare hashes during login

Authentication Flow

- 1 User enters password
- 2 System computes:
- 3 Compare with stored hash

$$H(\text{password})$$

Never Store Plaintext Passwords II

If the database leaks:

- attackers should not immediately recover passwords

Modern Password Storage

Older systems:

- UNIX `/etc/passwd`
- simple one-way transformations

Modern best practice:

- use adaptive password hashing algorithms:
 - bcrypt
 - scrypt
 - Argon2
- intentionally make password verification expensive

Goal

Increase the cost of large-scale password cracking attacks.

Slowing Down Guessing Attacks

Online attacks can be rate-limited.

Common defenses:

- delay after failed login attempts
- exponential backoff
- temporary account lockout
- CAPTCHA challenges
- anomaly detection

Example

After 5 failed attempts:

- require MFA
- introduce long delays
- notify the user

These defenses dramatically reduce the feasibility of brute-force attacks.

Password Salting I

Problem

Without salting, identical passwords produce identical hashes.

Attackers can:

- crack many accounts simultaneously
- use precomputed rainbow tables

Solution: Salt each password

$$\text{hash}(\text{password} + \text{salt})$$

Store:

$$\langle \text{userID}, \text{salt}, H(\text{password} + \text{salt}) \rangle$$

Benefit

Each account must be attacked independently.

Password Reset Security

Password resets are authentication mechanisms.

If password reset is weak, the entire authentication system is weak.

Common weaknesses:

- weak security questions
- SMS interception
- phishing
- email account compromise

Modern recommendations:

- multi-factor recovery
- recovery codes
- hardware security keys
- suspicious activity monitoring

Implementation Details Matter

Security failures are often implementation failures.

Examples:

- timing attacks
- insecure random number generators
- improper error handling
- outdated cryptographic algorithms

Historical lesson:

- even well-designed systems can fail due to subtle bugs

Heartbleed (2014)

OpenSSL implemented the TLS “heartbeat” extension incorrectly.

A missing bounds check allowed attackers to request more data than intended and read arbitrary memory from servers.

Attackers could steal:

- user passwords
- session cookies
- private TLS keys
- confidential server data

Although TLS and modern cryptography were theoretically secure, a tiny implementation bug compromised millions of systems worldwide.

Limits of Passwords

Even with strong engineering practices, passwords remain vulnerable.

Persistent problems:

- users still choose weak passwords
- password reuse across websites
- phishing attacks
- credential stuffing
- social engineering

Credential Stuffing

Attackers reuse leaked username/password pairs from one breach against many other services.

Social Engineering and Phishing

Key Observation

Attackers often target humans instead of cryptography.

Social engineering:

- manipulating people into revealing secrets

Phishing:

- fake login pages
- fraudulent emails or messages
- impersonation of trusted organizations

Modern phishing campaigns are:

- automated
- highly personalized
- AI-assisted

Two-Factor Authentication (2FA)

Idea

Authentication should require evidence from multiple independent categories.

Three classic authentication factors:

- ① Something you know
 - password, PIN
- ② Something you have
 - phone, smart card, hardware token
- ③ Something you are
 - fingerprint, face, iris, voice

Why Multi-Factor Authentication Works

Key Insight

Stealing one factor should not be enough.

Examples:

- password + authenticator app
- password + hardware security key
- smart card + PIN
- SSH private key + passphrase

Modern MFA technologies:

- TOTP applications
- push notifications
- FIDO2 / WebAuthn security keys
- biometrics

Time-Based One-Time Passwords (TOTP)

Example login process:

- 1 User enters username and password
- 2 User enters temporary code from authenticator device/app

How It Works

Authenticator and server share:

- a secret cryptographic key
- synchronized time

Every 30 seconds:

$$\text{code} = f(\text{secret}, \text{time})$$

If the codes match:

- user likely possesses the device

Modern Authentication Trends

Current direction:

- move beyond passwords
- reduce phishing risk
- improve usability

Passwordless Authentication

Examples:

- passkeys
- hardware security keys
- platform biometrics

Major platforms now support:

- FIDO2
- WebAuthn
- device-bound credentials

Takeaways

- Passwords remain important but fundamentally imperfect
- Strong password storage is essential
- Salting and adaptive hashing slow attackers
- Humans remain a major attack surface
- Multi-factor authentication significantly improves security
- The industry is moving toward passwordless systems

Goal of MFA

Strengthen authentication by combining independent factors:

- something you know
- something you have
- something you are

Modern systems rely on cryptographic and device-based mechanisms rather than only passwords.

TOTP (Time-Based One-Time Passwords)

Examples: Google Authenticator, Microsoft Authenticator

Idea

Shared secret + current time $\rightarrow$ short-lived code

- Server and app share a secret key
- Code changes every 30 seconds
- No network connection required for generation

Security intuition: Even if an attacker sees a code, it expires quickly.

TOTP Limitations

- Vulnerable to real-time phishing (relay attacks)
- Secret key can be stolen if device is compromised
- Users can still be tricked into revealing codes

Key Insight

TOTP improves password security, but still relies on shared secrets.

Push-Based Authentication

Examples: Google Prompt, Microsoft Authenticator, Duo

Idea

User approves or denies a login attempt on a trusted device.

- Login attempt triggers notification
- User taps “Approve” or “Deny”
- Often includes context (location, device, IP)

Advantage: Very easy to use compared to entering codes.

Push Authentication Risks

- MFA fatigue attacks (repeated prompts)
- Users may approve malicious requests accidentally
- Phishing sites can trigger real login attempts in real time

Key Insight

Push MFA improves usability, but does not eliminate phishing risk.

Examples: YubiKey, platform passkeys (Apple, Google, Microsoft)

Core Idea

Public-key cryptography replaces shared secrets.

- Device generates public/private key pair
- Private key never leaves device
- Server stores only public key

Login process:

- 1 Server sends challenge
- 2 Device signs challenge
- 3 Server verifies signature

Why FIDO2 is Strong

- Resistant to phishing (bound to correct domain)
- No reusable shared secrets
- Cannot replay captured authentication data
- Often requires physical presence (touch or PIN)

Key Insight

Authentication becomes a cryptographic proof of possession.

Examples: fingerprint, face recognition, iris scan

Idea

Use physical or behavioral traits to verify identity.

- Measured locally on device
- Converted into feature templates (not raw images)
- Used to unlock cryptographic credentials

Biometrics: Security Properties

- Very convenient and fast
- Cannot be changed if compromised
- May have false positives/negatives
- Usually not used alone for high-security systems

Key Insight

Biometrics typically unlock a cryptographic key rather than replace it.

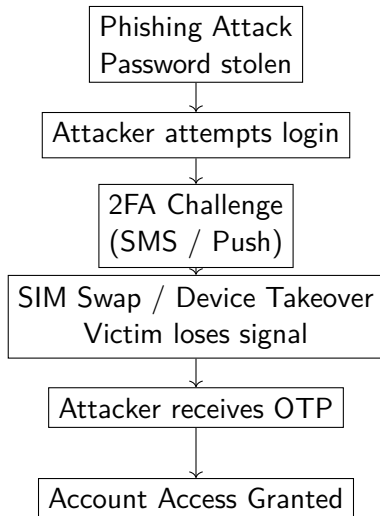
Summary of Modern MFA

- TOTP: time-based shared secret codes
- Push MFA: user approves login attempts
- FIDO2/WebAuthn: cryptographic challenge-response (strongest)
- Biometrics: local identity verification, often as unlock factor

Trend

Modern authentication is moving away from shared secrets toward device-bound cryptographic credentials.

Attack Chain: MFA Bypass in Practice



Real-World Case Study: Uber Breach (2022)

What happened?

A social-engineering attack bypassed multi-factor authentication and led to internal system compromise at Uber.

Initial access

- Attacker obtained employee credentials via phishing
- Password alone was sufficient to initiate login attempts

Key security control in place

- Push-based MFA (mobile approval prompts)

Uber Breach (2022): MFA Bypass Mechanism

Attack progression

- 1 Attacker repeatedly triggered login attempts
- 2 Victim received multiple MFA push notifications
- 3 Victim eventually approved a request (MFA fatigue / annoyance)
- 4 Attacker gained authenticated access

Key Outcome

Once MFA was approved, attacker gained access to internal systems, including communication and administrative tools.

Key lesson: Even correct MFA implementation can fail if users are socially engineered.